
*Bachelor Thesis Applied Computer Science*
(Academic year 2025—2026)

# Federated Learning Approach for Obstructive Sleep Apnea Prediction

Author:

Matej Vesel

Company:
Result d.o.o

Company mentor:
Marija Gorenc Novak, PhD
*Mathematical Statistics*

University mentor:
David Vandenbroeck, MA
*Communication Studies*

# Acknowledgements

Hvala Manja, ker si mi stala ob strani in me vodila skozi ta zaključni del študija. Tvoje zanimanje tega področja je tekom pripravništva in diplomskega dela prešlo tudi name in najbolj izrazit spomin, ki bo zagotovo ostal z menoj še dolgo, so vsakodnevna jutranja srečanja, kjer si prav vedno prišla ter začela dan z iskrenim nasmehom in tihim sporočilom, da nobena težava ni kos tistemu, ki služi s srcem in zadovoljstvom.

Pravtako se zahvaljujem Davidu, ki mi je na mnogotera vprašanja glede pripravnštva in diplomskega dela odgovorajal že celo leto, čeprav je bil moj mentor zgolj zadnji mesec.

Hvala mama in hvala oče, ker sta stala z menoj toliko let in me podpirala. Omogočila sta mi, da sem prišel do študija, ki sem si ga želel in ga s tem delom tudi zaključujem. Hvala za vso vajino ljubezen.

Za radoživost, igrivost, navihanost in ljubezen se zahvaljujem tudi svojemu bratcu ter punci, ki me je (tudi) tekom tega dela velikokrat razveselila s kakšno poslastico oziroma pojedino, za katero si sam ne bi vzel časa.

Thank you Manja for standing besides me and guiding me through this final part of the study. Your interest in this field has passed through me during the internship and bachelor thesis and the most prominent memory that will surely stay with me for a long time were the daily morning meetings, where you always came and started the day with an honest smile and a silent message that no trouble can match the one that serves with his heart and contentment.

Likewise I am thanking David, who has been answering my numerous questions regarding internship and bachelor thesis for a whole year, although he has been my mentor only for the last month.

Thank you mother and thank you father, because you have been standing with me for this many years and have supported me. You made it possible for me to come so far to start this study that I have wished for and which I am also finishing now with this work. Thank you for your love.

For joyousness, playfullness, mischievousness and love I am also thanking my little brother and girlfriend, who has (also) during this work delighted me many times with a delicacy or feast, respectively, for which I would not have taken time by myself.

# Abstract

Machine learning requires large amounts of data. The challenge is not a lack of data, but that most of it is private and cannot be moved or disclosed. Machine learning typically requires all data on a central location, however, private data must often remain at its source. In this thesis I explore an alternative machine learning approach where instead of moving the sensitive data to a central machine, we move the "knowledge" (technically model weights) to the edge devices, in other words, closer to the origin of data where it is kept under all security measures, for example a hospital. The machine learning in the thesis is done in a healthcare context and I implement federated learning for diagnosis of obstructive sleep apnea (a sleep-related breathing disorder) by dividing the learning process into three hospitals that do not share any sensitive patient data. Finally, I compare the federated approach with a centralized baseline.

## Contents

# 1 Scope & Vision

## 1.1 Background

The project was carried out at Result d.o.o. It is a software development company, but also with a focus on data, artificial intelligence, and business solutions. Result works on many projects, some of which are European. One such is eEarly — an open platform for analysis and gathering of health data **EEarly**. It also provided the context for this bachelor thesis.

The European Union has many regulations and laws related to data, and there are even more for each individual country, organisation, and so on. This introduces challenges for projects like eEarly that utilize artificial intelligence and deal with sensitive medical data. The motivation for my bachelor thesis was to find the bridge between sensitive data, artificial intelligence, and a complex regulatory landscape.

The aim of this work was early diagnosis of sleep-related breathing disorders by analysing biometrical data. The main sources of data were APNEA HRV+SPO2 DATASET **julia-serdaAPNEAHRV+SPO2DA** and a smart medical ring Wellue O2Ring[1] that a person can wear while it continuously performs medical measurements. The artificial intelligence model used heart rate and oxygen saturation (SpO2) to retrospectively label patients' measurements and indicate periods consistent with a sleep-related breathing disorder called obstructive sleep apnea (OSA) **strolloObstructiveSleepApnea1996**.

## 1.2 Problem statement

In an ideal situation, an artificial intelligence model could train on all data without restrictions. In reality, besides strict European Union laws, such as General Data Protection Regulation (GDPR) **sasirekhaSystematicReviewPrivacypreserving2025**, **prayitnoSystematicReviewFederated2021**, each hospital normally keeps its data isolated. This reveals the main issue: An artificial intelligence model can — without considerable legal and data privacy effort — only train on one hospital. Consequences of this are poor model performance with less accurate predictions, and the new technologies are not being utilized fully **abdallaFutureArtificialIntelligence2025**. From this follows the proposal, which is not to loosen the law, but to explore techniques like federated learning **FederatedLearning2026** to satisfy both the regulations and the artificial intelligence's demand for data.

## 1.3 Intention

The main intention of the project is to develop a federated learning proof-of-concept (POC) for the company, which can later be used as a starting point for other projects and might be fully implemented, possibly in a healthcare environment. It might also become a new privacy-oriented artificial intelligence solution that the company could offer to clients for commercialization as a reusable asset.

## 1.4 Scope within the bachelor thesis (MVP – need to have)

The minimal scope includes a simulation of federated learning on a single machine only with Python and without any containerization such as Docker. The scope can be seen on the Table **??** on page **??** with additional details in Appendix **??**. The requirement descriptions include terminology introduced in Section **??**. "Must have" features describe the minimal viable product (MVP); without these, the project is not deemed finished, unless blockers are clearly stated.

## 1.5 Ultimate scope (want to have)

The prototype in its most complete form would be a simulation of federated learning on four machines — one global model and three hospitals — using a containerization technology like Docker, and the machines would communicate from separate networks with security features such as TLS and differential privacy. Ultimate scope of the project is further defined with features of "Should have" and "Could have" importance in the Table **??** on page **??**. As importance lowers, features become closer to those of realistic setting in which federated learning could be deployed, and more unknown challenges might appear.

[1] https://getwellue.com/pages/o2ring-oxygen-monitor

# 2 Software Requirements Specification

## 2.1 Target group

The primary target group of the project is developers. Most important aspect to consider for this target group is the knowledge to read code and its documentation. Their needs are not only to be able to understand code and its documentation, but also know how to interpret the results at the most basic level, and most important, they need to know how to run the project.

For the umbrella project eEarly, there are more potential stakeholders such as patients, whose sensitive data is guaranteed to remain private during the artificial intelligence training process, and doctors that would receive improved disease predictions. Perhaps there are even stakeholders from the legal perspective since the sensitive data is not moved and there is less possibility to breach current legislations and less need to adapt it. Though that is out-of-scope for the federated learning project.

## 2.2 Method

In order to discover, track, and prioritize all requirements, I was, especially in the beginning, asking a variety of questions to my mentor at Result d.o.o. and kept sorting them by importance using the MoSCoW prioritization framework[2]. Besides being very simple and clear, this method has proven itself in my previous projects, which is the main reason why I chose it as my main tool at this stage. All of this was done on Jira in a kanban board (see Appendixes **??**, **??**, **??**), so that it has always remained organized and visible to me and mentors.

## 2.3 Requirements

*Functional requirements* can be seen in the Table **??**. "Must have" requirements form the minimal viable product (MVP). Out of all requirements, I have created features that can be seen in the list below.

1. As a developer,
   I want to run a single-machine and non-containerized simulation of apnea predictions using federated learning,
   so that the whole structure is as simple as possible.

2. As a developer,
   I want to run a single-machine and containerized (with Docker) simulation of apnea predictions using federated learning,
   so that the whole project can be easily started by anyone on any machine.

3. As a developer,
   I want to run a multi-machine and containerized (with Docker) simulation of apnea predictions using federated learning,
   so that the structure comes closer to real and production-ready implementation.

4. As a developer,
   I want to run a multi-machine and containerized (on Kubernetes) simulation of apnea predictions using federated learning,
   so that the structure comes closer to real and production-ready implementation.

5. As a developer,
   I want to have a differential privacy mechanism implemented on top of federated structure,
   so that I can assure the system also secures weights and parameters that are sent to a central server.

*Non-functional requirements*:

- A developer needs to understand
  - the code,
  - the documentation,

[2] https://community.atlassian.com/forums/App-Central-articles/Understanding-the-MoSCoW-prioritization-How-to-implement-it-into/ba-p/2463999

– how to interpret results at the most basic level, and

– how to run the project.

Table 1: Functional requirements with their importance, higher rows correspond to higher priority. For a further dissection of minimal viable requirements, see Appendix **??**

| **Importance** | **Requirement** |
|---|---|
| Must have | Research of the technology and alternatives. |
| Must have | Preparation of development environment. |
| Must have | Single-machine and non-containerized simulation of apnea predictions using federated learning. |
| Should have | Single-machine and containerized (with Docker) simulation of apnea predictions using federated learning. |
| Could have | Differential privacy mechanism implemented. |
| Could have | Multi-machine and containerized (with Docker) simulation of apnea predictions using federated learning. |
| Could have | Multi-machine and containerized (on Kubernetes) simulation of apnea predictions using federated learning. |
| Will not have | Non-basic visualizations. |
| Will not have | Advanced artificial model. |
| Will not have | Other security features like secure multiparty computation. |
| Will not have | Production ready features, e.g. blockchain technologies like swarm learning or compatibility with NVIDIA FLARE. |

# 3 Preliminary Research

## 3.1 Federated learning

Imagine there are researchers working on a project and they do not tell each other anything about themselves, their names, education, age, and so on. They only contribute with their current knowledge without revealing their personal information. This is what federated learning tries to achieve **FederatedLearningCollaborativea**. Later on I talk about global and local models, in the example I just gave, researchers play a role of local models, and the project would be a global model.

Federated learning is a *decentralized learning* technique, which means the learning is not accomplished centrally on a single device — normally a server — but is kept on the edge, close to the main source of the data. Since it is decentralized, this also leads to great benefits regarding *privacy* and regulations as data is not moved out of its safe zone. In practice, we accomplish this using two types artificial intelligence models: global and local. The algorithm **strykerWhatFederatedLearning2025** contains the following steps:

1. Global model is initialized.
2. Global model sends itself to all the local models. In a medical example, local models would be hospitals.
3. Local models train on sensitive (medical) data. There is no raw data exchanged between *any* of the models.
4. Local models send the weights back to the global model. Image that they share their knowledge, without the data.
5. Global model combines the weights — usually by averaging them —, and becomes better at predicting diseases, for example.
6. The process repeats from the second step. Each local model will now become better as it will receive the average "knowledge" from all others.

### 3.1.1 Types

There are two main types of federated learning that can be implemented based on the similarities between datasets of clients. A definition of a dataset is essential: A *dataset* is a collection of data, commonly structured into a format comparable to a spreadsheet where each row represent a record (also called example). A record consists of values that represent either a feature or a label. A feature is input for a machine learning model, such as heart rate. A label is either output of a machine learning model — in disease prediction, labels would be 0 ("Obstructive sleep apnea not detected") or 1 ("Obstructive sleep apnea detected") — or an answer for a machine learning model while it is being trained on so called labeled data **MachineLearningGlossary**.

**Horizontal Federated Learning** In horizontal federated learning, clients' datasets describe same features meanwhile the *data is different* **prayitnoSystematicReviewFederated2021**, **zhangIntroductionFe**
**FederatedLearningWhat**, **strykerWhatFederatedLearning2025a**. For distinction with vertical federated learning, it is important here to keep in mind an almost obvious fact — By saying "the data is different" between clients, we mean that the feature that uniquely identifies a record, such as an email or a national identification number, is never duplicated across clients' datasets. For example, there are most likely many hospitals worldwide that have treated sleep apnea and have targeted same medical features, such as heart rate and oxygen saturation, but collected different data for different patients (see Figure **??**). A federated learning model could predict sleep apnea based on all hospitals' client models.

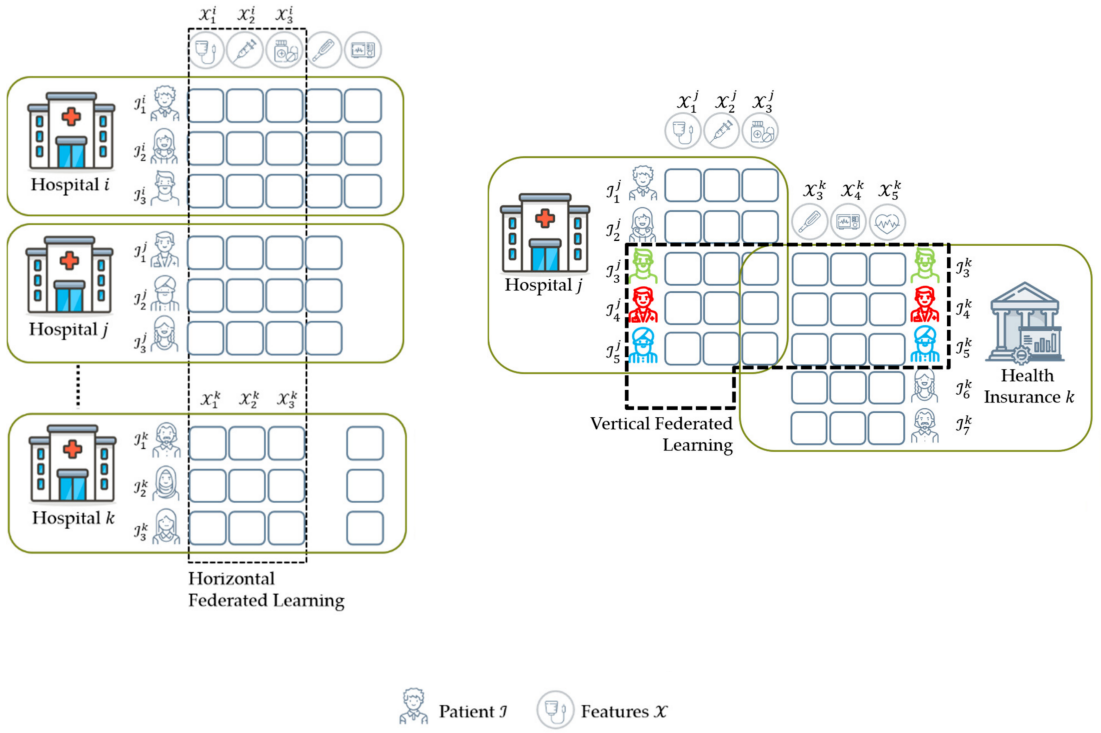
**Vertical Federated Learning** In vertical federated learning, clients' datasets describe different data and different features, *except for identification* features such as identification number **prayitnoSystematicReviewFederated2021**, **zhangIntroductionFederatedLearning2022**, **FederatedLearningWhat**, **strykerWhatFederatedLearning2025a**. For example, a country has a hospital for sleep disorders that collects features like patient national identification number, heart rate and oxygen saturation, and a hospital for cancer that collects patient national identification number, x-ray scan images and blood information (as visualized in Figure **??** with a small difference that a health insurance is used instead of a cancer hospital) **shiVerticalFederatedLearning2025**.

A federated learning model could connect the knowledge of both hospitals using patient identifier, then find patterns, and become more holistic in its predictions.

**Federated Transfer Learning** In federated transfer learning, clients' datasets describe different data and different features, including the identification features **prayitnoSystematicReviewFederated2021**, **zhangIntroductionFederatedLearning2022**, **FederatedLearningWhat**, **strykerWhatFederatedLearning20** For example, two countries have two hospitals and there is very little overlap between features or data — the patients' identification is also mostly different — yet federated transfer learning can be applied in this scenario to combine their knowledge.

We will focus on horizontal federated learning prototype, because our features remain same across clients, namely hospitals.

Figure 1: Visualization of main types of federated learning in healthcare scenarios, sourced from **prayitnoSystematicReviewFederated2021**. On the left horizontal federated learning from different data with same features. On the right vertical federated learning from different data and mostly different features.




## 3.2 Related projects

Numerous similar projects with alike problems can be found, though there are some more noticeable and important.

In 2017, Google introduced federated learning while implementing the "Next word suggestion" feature on their Gboard. They took user interaction as their input and trained an artificial model to predict users next word. Without this innovating project, all the private typing data from millions of devices would have to be transferred to a central server. At this point, we can easily understand Google's main concerns of standard data pipeline: Bandwidth usage and latency **mammenFederatedLearningOpportunities2021a**, **FederatedLearningCollaborativea**.

During COVID-19 pandemic, researchers all over the world established connections and collaborated to create an artificial intelligence platform that predicts whether you have the disease based on CT (X-ray) chest scans. This was an initiative called Unified CT-COVID AI Diagnostic Initiative (UCADI), and the reason I am mentioning it here is that they made it all possible by using federated learning **mammenFederatedLearningOpportunities2021a**, **baiAdvancingCOVID19Diagnosis2021**.

## 3.3 Related solutions

Main characteristics of federated learning described in Section **??** are decentralized learning and privacy; I use this two for finding related solutions below.

### 3.3.1 Decentralized learning techniques

*Fog computing* is an improved approach on top of federated learning. It originates from the network challenges of federated learning, focuses on layered architecture of communication between local and global model, and aims to reduce network traffic **gorrichoMASTERSDEGREETELECOMMUNICATIONS**

Being a decentralized technique, federated learning still has a central, global model that aggregates all parameter updates. *Gossip learning* — also called distributed federated learning (DFL) **shammarSwarmLearningSurvey2025** — is even simpler in this sense, as it does not have a global model. The updates and aggregations are all taken care by the local models themselves **hegedusDecentralizedLearningWorks2021b**.

Similar to gossip learning is *swarm learning*. The main difference is its use of blockchain peer-to-peer communication that is configured between the local models. This enables swarm learning to be a very related solution, with even better potential privacy and fault tolerance (because there is no single point of failure like the global model server) **shammarSwarmLearningSurvey2025**.

Centralized learning does not come into considerations because the movement of data to a central server is restricted by data regulations. Federated learning is well suited for the MVP, as it is simple and is the core concept behind all decentralization alternatives above that introduce unnecessary complexity for the project at this stage, although techniques like swarm learning would be welcome on a production-ready implementation.

### 3.3.2 Privacy preserving techniques

In general, *homomorphic encryption* creates a possibility to make operations on encrypted data, without ever decrypting it. This can be applied both to normal and federated machine learning, although it is rather an addon than a replacement for the latter. The challenges include computational overhead and lack of support for some operations, thus finding a balance between privacy and performance is an important question **hallajiDecentralizedFederatedLearning2024**, **marquesTrainingMachineLearning2026**.

*Secure multiparty computation* allows to split private data from many participants, share it in this split form to others, and then let any participant compute (public) functions, such as average and maximum without knowing any other participants' private data **evansPragmaticIntroductionSecure**. In federated learning, sharing weights with the central server still brings up some privacy concerns that can be handled by applying secure multiparty computation **mugunthanSMPAISecureMultiParty**. This adds some computation cost, along with even more communication overhead **hallajiDecentralizedFederatedLe**

*Differential privacy* introduces random noise to the weights sent during federated learning while still maintaining the statistical properties of a dataset **dankarPracticingDifferentialPrivacy2013**. This guards the system from inference and poisoning attacks. For example, *inference attack* would mean that a middleman can infer whether a person has a certain disease from the outputs of the global model **wuInferenceAttacksTaxonomy2024**; *data poisoning attack* would mean that a middleman intercepts and modifies the weights sent to the global or local model, making the predictions of the model less accurate or corrupted **gabrielliSurveyDecentralizedFederated2023a**.

All privacy alternatives mentioned above are strongly related, but optional for the scope of this work. They clearly point out main weak parts of the federated learning which have to be kept in mind in phases later on.

## 3.4 Required technology

There were minimal hardware requirements to develop the most important parts of the project. Must have functional requirements were satisfied only by using a single computer, and a dedicated remote server with a strong computing capabilities. It was specifically meant for faster model training, although it was not a necessity. Later on, another computer was added to configure the federated network in a multi-machine environment, making it more realistic. A smart ring sensor (Wellue O2Ring) was considered as a data source, but decided to be out of scope for this thesis and not used at the end.

Software requirements consisted mostly of a programming language, integrated development environment and a virtualization technology. Since the apnea prediction model was already coded in a programming language Python (with Tensorflow machine learning library), I made sure this

was also the case for the federated learning network. Federated Flower network was coded in Python, specifically configured for use with Tensorflow and a remote procedure communication protocol gRPC. Coding itself was done in IntelliJ and Visual Studio Code; the latter was used for easier SSH access to the remote machine learning server mentioned in the previous paragraph. Virtualization technology used after developing the minimal viable product was Docker, which made it possible to run all necessary code on separate computers without any other software and with additional TLS security. No other programming language or software was used or tested during this project.

## 3.5 Comparative study

From **braungardtFlowerPySyftCo2024**, **yurdemFederatedLearningOverview2024**, **riedelComparativeAna**
I have extracted four main open-source frameworks suitable for the project: TensorFlow Federated (TFF), NVIDIA Federated Learning Application Runtime Environment (NVIDIA FLARE), and FedML. The latter had vastly different opinions among papers, so I did not take this framework into consideration.

**Flower** It's main characteristic is easy integration with different deep learning frameworks, and that it offers a beginner- and prototype-friendly entrance into federated learning **mehdiComprehensiveReviewOpen**
**yurdemFederatedLearningOverview2024**, **rajeshkumarTop10Federated2026**, **guptaBenchmarkingFedera**
As Flower was new in 2025, research papers from that year showed it has lacked community support and documentation **mehdiComprehensiveReviewOpenSource2025**, **yurdemFederatedLearningOverview202**
which has changed the next year, and is now apparently considered to have strong documentation **rajeshkumarTop10Federated2026**. It is especially well-suited for edge IoT devices and production ready **mehdiComprehensiveReviewOpenSource2025**, although additional privacy features have to be engineered manually **rajeshkumarTop10Federated2026**, unlike with frameworks like NVIDIA FLARE.

**NVIDIA FLARE** It's main characteristic is enterprise-readiness **mehdiComprehensiveReviewOpenSource202**
**cotocusTop10Federated2026**, **rajeshkumarTop10Federated2026**, **guptaBenchmarkingFederatedLearning**
with many built-in features, especially for privacy **cotocusTop10Federated2026**. It is overall very good in all aspects (also documentation) **braungardtFlowerPySyftCo2024**, **mehdiComprehensiveReviewC**
which is the reason it might seem heavy for smaller projects **rajeshkumarTop10Federated2026**.
The main disadvantages is that you are restricted to the NVIDIA ecosystem **mehdiComprehensiveReviewOpenSou**

**TensorFlow Federated** It's main characteristic is that it is most widely used, but complex, only compatible with TensorFlow, and can be integrated into the least environments **mehdiComprehensiveReviewOpen**
**yurdemFederatedLearningOverview2024**.

The choice comes down to Flower or NVIDIA FLARE. Luckily, these two have recently announced collaboration and compatibility between each other **rothSuperchargingFederatedLearning2024**, **saysSuperchargingFederatedLearning2025**, **laneAnnouncingNVIDIAFlower2024a**, meaning Flower can be integrated into the NVIDIA runtime environment and immediately leveraging enterprise-ready environment from NVIDIA and simplicity from Flower. For this reason, Flower will be used to create the prototype.

## 3.6 Final selected technology / project tools

Project management tools used throughout this bachelor thesis where Jira, which I used for tracking work items and requirements, and Microsoft Excel, which I used for time tracking and diary of what I did each day. More details were described in Section **??**.

Me and my coach respected agile principles, and the flow of the project followed scrumban development methodology — a combination of scrum and kanban. Since I was the only developer, most scrum events were skipped, although I did have daily meetings with my company coach. The way tickets were tracked is completely based on kanban, having a board with todo, doing, and done columns.

## 3.7 Choice reflection

Decisions described above were based on experience, research and communication with my company mentor. The technologies were correct, which was proven by the fact that we did not have to

change any during the development, especially Flower that truly turned out to be well-documented and beginner-friendly, meanwhile also supporting advanced enterprise requirements with documentation including additional security techniques and many large-scale integration possibilities, two of them being Docker and Kubernetes. For me, it remains a question, what weak points would be revealed during its complete deployment in a production setting. The project tools were great, especially Jira that offers a powerful way to increase productivity and insight into the project organization and status, without unnecessary features.

# 4 Reflection

## 4.1 General reflection

### 4.1.1 Achieved objectives

The project requirements written in Table **??** turned out to be well-thought as majority was implemented. By successfully establishing single-machine non-containerized simulation of apnea predictions minimal viable product was delivered. This was further more extended — using Docker — to multi-machine containerized setup. On top, additional security was configured with TLS certificate requirement between the all communicating nodes. Although this security might not be necessary since it was done in a single network, the federated setup is ready to be tested in a multi-network environment where this would become a must. All requirements with associated completion status can be seen in Table **??**.

Table 2: Functional requirements with their importance and completion status, higher rows correspond to higher priority. For a further dissection of minimal viable requirements, see Appendix **??**

| **Importance** | **Requirement** | **Implemented** |
|---|---|---|
| Must have | Research of the technology and alternatives. | Yes |
| Must have | Preparation of development environment. | Yes |
| Must have | Single-machine and non-containerized simulation of apnea predictions using federated learning. | Yes |
| Should have | Single-machine and containerized (with Docker) simulation of apnea predictions using federated learning. | Yes |
| Could have | Differential privacy mechanism implemented. | No |
| Could have | Multi-machine and containerized (with Docker) simulation of apnea predictions using federated learning. | Yes |
| Could have | Multi-machine and containerized (on Kubernetes) simulation of apnea predictions using federated learning. | No |
| Will not have | Non-basic visualizations. | No |
| Will not have | Advanced artificial model. | No |
| Will not have | Other security features like secure multiparty computation. | No |
| Will not have | Production ready features, e.g. blockchain technologies like swarm learning or compatibility with NVIDIA FLARE. | No |

However, there are still some unresolved issues. One of them being the aggregated evaluation metrics showing presumably wrong results when it comes to precision and recall scores — these are always zero, in contrast to centralized training where this does not happen. On the other hand, the train metrics are correct, which is why the task is deemed achieved, besides that my focus was not on improving model performance. Perhaps testing the final model on all data would be an indicator about the direction of the issue — were the metrics not aggregated correctly, or is there an issue solely in the evaluation code.

A summary of final metrics from both centralized and federated approaches can be seen in Table **??**. The differance between these two machine learning techniques, in terms of metrics, turn out to be very minor, around $0.03 \pm 0.05$, with federated learning mostly having slightly lower values; except for the test metrics, where we suspect a minor bug in the code. Results of another test can be found in Appendix **??**, all tests and project code can also be found on GitHub[3].

### 4.1.2 Test framework / technique

No automated testing was written during the project. The model is manually tested by using evaluation and prediction scripts, which isolate some patients' measurements from the training procedure and later on execute predictions on them to give us representative results of model performance. The federated framework part of the project also does not have any special tests. Important to note is that the different hospitals, in different Docker containers do not share any patients' measurements data during training and testing, which is also the goal of federated learning.

In production, besides automated testing of data pipeline, privacy that federated learning advertises should be thoroughly tested not to show any vulnerabilities of attacks described in Section **??**, such as inference attack.

[3] https://github.com/OP4LIFE/federated-learning-code

Table 3: Results of centralized and federated learning approach and their comparison. In a federated setting, the test consisted of 3 hospitals (clients), 5 rounds and 15 epochs. In a centralized setting, it consisted of $15 \cdot 5 \cdot 3$ epochs (255 epochs). Hyperparameters were the same for both approaches. Number 0 or 1 after metric name indicates whether it was calculated for patients with or without obstructive sleep apnea.

| **Metric** | **Centralized** | **Federated** | **Difference** |
|---|---|---|---|
| train_accuracy | 0.80 | 0.79 | 0.01 |
| train_f1_1 | 0.52 | 0.51 | 0.01 |
| train_f1_0 | 0.78 | 0.74 | 0.02 |
| train_precision_1 | 0.43 | 0.40 | 0.03 |
| train_precision_0 | 0.86 | 0.87 | 0.01 |
| train_recall_1 | 0.64 | 0.70 | 0.06 |
| train_recall_0 | 0.72 | 0.65 | 0.07 |
| test_accuracy | 0.76 | 0.83 | 0.07 |
| test_precision | 0.35 | 0.00 | 0.35 |
| test_recall | 0.48 | 0.00 | 0.48 |
| avg_train_difference | | | 0.03 |
| avg_test_difference | | | 0.3 |
| avg_difference | | | 0.111 |

#### 4.1.3 Security

The whole concept of federated learning is build for the purpose of securing sensitive data by not sharing it to third parties, which is also what has been implemented in this proof-of-concept. In a multi-machine setup, Docker containers do not share any private patients' measurement data between each other. They merely exchange the knowledge in terms of model weights. Most additional security features mentioned in Section **??** were left out in favor of expanding the project towards the infrastructural direction by containerizing the implementation. While developing that, secure TLS communication was enforced in order for the gRPC communication between nodes to become encrypted and weights cannot be directly read by a middleman.

Possible improvements in the project's security mostly entail differential privacy, which is already supported by Flower out-of-the box and offers a mostly straight-forward implementation. More difficult security features to further research and develop would be secure multiparty computation and homomorphic encryption. In case of progressing the project to Kuberenetes, it is important to understand how it can negatively affect the security that Flower brings and use an appropriate framework such as kubeFlower **parra-ullauriKubeFlowerPrivacypreservingFramework2024**.

#### 4.1.4 Improvements

Starting with a non-functional improvement: readability. The model code I took was a few months old and there is a more readable and clean model that a colleague at the company has developed since. Plugging in that model would be a highly beneficial next step. Afterward, running the federated network using docker on *separate* networks would be one step closer to real implementation. Adding TLS certificates issued by an official authority like Let's Encrypt, instead of using self-signed certificates like I have done now, is a mandatory move; Let's Encrypt also offers test certificates, which could be great while exploring this part of security. Finally, reducing bandwidth and model weight size sent over network even more might be an interesting topic of research.

#### 4.1.5 Added value of the project

This prototype brings great flexibility to developers as they can swiftly gain understanding of the clear structure that Flower provides, they can adapt the code for other purposes due to its modular nature — for example by switching to a different model for a different scenario — and all of that, along with containerization, makes it possible to easily scale the prototype to a business-ready solution.

The greatest added value is ultimately for the patients, or any kind of clients whose data is used for the system. Besides better and faster diagnosis, most importantly, federated learning brings privacy. Privacy enables trust and there is no business without trust. This is the key impact that this approach brings compared to standard machine learning.

Beside patients, doctors receive improved tools, project managers and lawyers have less regulation burden, and scientists, together with the whole research community, can explore new solutions with the help of additional data that federated learning brings. Data scarcity is a problem with machine learning, especially in healthcare. Imagine that European hospitals create a federated network around the sensitive data, giving researchers the possibility to switch and explore different machine learning models with sufficient and realistic data, and with respect to each person's records.

### 4.1.6 Future opportunities and next steps

This project could be potentially adapted to any other machine learning project, especially those with privacy concerns or many IoT devices, sensors, moving devices, and so on. It could be used in many optimization scenarios, such as improving car safety features by learning on a fleet of vehicles without directly sending the data over the network, enhancing traffic light systems to reduce traffic jams, just to name a few possibilities. Mastering the implementation of a federated system might bring a company such as Result d.o.o a wide range of real-world project opportunities.

## 4.2 Personal reflection

The project was planned well and the development went mostly smoothly. There are two small bits I would change if I did it again: (1) The remote server for machine learning that I had available holds a powerful graphics card. Knowing the difference in training speeds, I did my best to make it work on the federated training process. After having spent around a day if not more and with the help of three other employees, Python and Tensorflow successfully recognized it. The training was blazing was! Until Out Of Memory errors started to appear and through investigation, it was clear that the heavy apnea prediction model, simulated on three clients/hospitals, demanded somewhat more than 24 GB of available virtual RAM, which the powerful graphics card offered. This kept happening despite using a minimalistic dataset. Perhaps adding another such card to the server would solve the problem and was however, but it was not reasonable, timewise. Switching back to CPU-only training there were no further hardware issues. (2) There was some notion of possibly getting live measurements data from a medical ring and making on-the-fly predictions, although we all knew that it is out of scope and would require almost a miracle or a big underestimation of requirements' implementation time. Making live ingestion pipelines is a whole other scene, so I should have never even thought about it and clearly mark it with "Will not have" importance.

**Impact on people** Thinking about the impact of the matter on the people, it will most likely not bring as much notice as artificial intelligence did, as it is just one part of it. Although the bare concept can be very easily explained to almost any person and one can make sense out of it on its own, while understanding the fact that their sensitive data will remain stationary and under all regulations *as it has used to*, without any artificial intelligence model directly looking and processing it. This would give people less fear of artificial intelligence stealing their data, knowing that it will remain under protection of their hospital as before.

It is also worth to remind ourselves about the prediction bias that artificial intelligence models often cause. Whatever the source reason of the bias is, I have not done research on how federated learning concept might solve it (or make it worse), expect for the fact that with it, we can explore vast amounts of unrevealed private data and perhaps minimize bias that way.

Another impact the tool might have on people is that by knowing companies can use it to learn on your private data, one can feel even more pressure and become unsure about what is really going on and when he agreed with all of that. For example, I doubt most users of Google's Gboard smartphone keyboard know federated learning concept is behind its next word suggestion feature **hardFederatedLearningMobile2019**, and that the suggestion knowledge is retrieved from someone else's private typing that contains who knows what. It might be beneficial to think about it from another perspective: A doctor also has lots of practical experience and has agreed not to share any personal data about his patients with others, although he has the right to treat any patient with all of his knowledge; this is exactly what federated learning concept tries to achieve.

**Impact on society** Federated learning also has impact on society. Looking at it from the view of *economy*, it brings profit, since it can make better models with bigger data input and less bandwidth cost. From a *policy* perspective, it is a more compliant approach with less regulation burden. The approach has especially important impact on *healthcare*. It enables very good and early personalized diagnosis possibilities, but more valuable is the *foundation* it creates for artificial

intelligence applications and research. When inspecting it from an *educational* standpoint, there should not be much of an impact, except perhaps for special education tools built on machine learning, for example flashcard app Anki[4] that uses Free Spaced Repetition Scheduler machine learning algorithm to find memory patterns **DeckOptionsAnki**, **2023˙OptimizingSpacedRepetition**. Imagine Anki's spaced repetition scheduler built using federated learning approach!

**Impact on climate** The federated learning project can also have impact on climate. With its low bandwidth, it improves machine learning climate footprint, and aids in creating more efficient processes for practically anywhere to enable energy reduction. For example, federatively learning car driver's patterns to optimize driving parameters in the car's computer, reduce risks of accidents and fuel consumption **luCrossVehicleFederatedLearning2025**. Another example would be weather predictions, where we receive millions of measurements from sensors and satellites. This data could be preprocessed on the device itself and propagated through a federated network.

As a final thought of reflection, the interest to learn more about machine learning field, especially its optimization capabilities, has grown during the project and will stay with me later on. I have never worked much with machine learning system, and now I see how much there is to learn and to gain.

## 4.3 Feedback

The integrated bachelor project was a wonderful experience and a perfect way to establish a strong connection with the industry and such good companies as Result d.o.o. I am especially glad my company mentor has kept in mind my profile — my experience is more in software development and infrastructure rather than in artificial intelligence — and arrange the traineeship to also have some machine learning, specifically hyperparameter tuning, in order for me to obtain some background and context before starting the bachelor project.

From the university side, it was also very well organized. The intermediate assignments were not mandatory, which was a great move to keep a student specially aware of the timeline, but not completely stressed about the deadlines. There are further possible improvements I can extract from the whole process of this bachelor thesis: (1) Introduce students to the powerful citation tools like Zotero. It is not completely obvious, but surely mandatory, and was not done in other courses before — which could be possible, for example in IT & Innovation where the reports and citations were already crucial. (2) The research part of this document took most of the time, which meant less for implementation. If I am correct, there are research and development bachelor project types. In case that's right, perhaps also splitting the templates for each specific type would be worth a thought. (3) The person responsible for integrated bachelor project, in my case David Vandenbroeck, also my mentor, has warned me before going abroad that the individual integrated bachelor projects normally take more time and effort. Keep warning students about that, as it is very important, also in the planning phase to know that the scope could seem almost poor to be truly realistic.

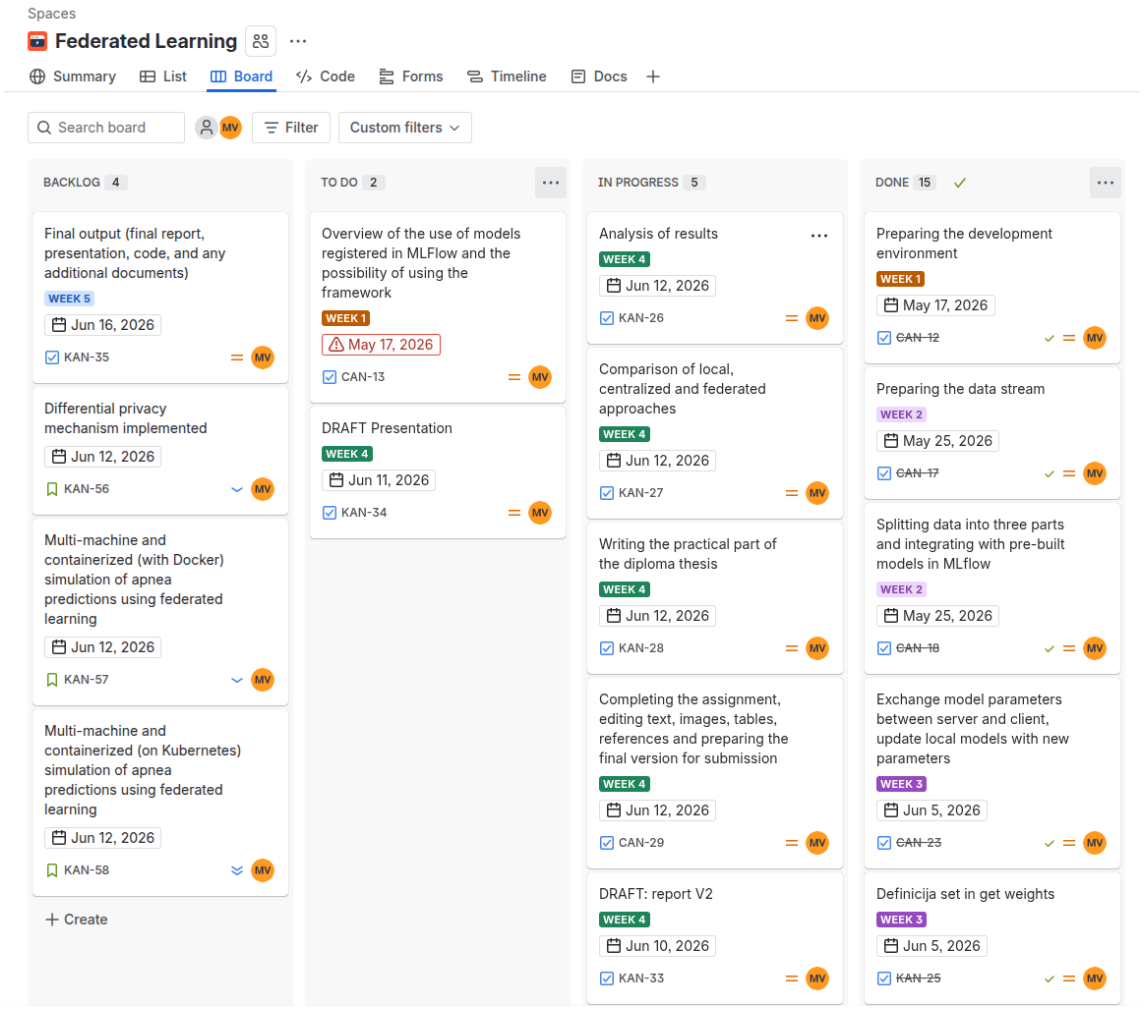
[4] https://apps.ankiweb.net/

# Appendices

## A Story Mapping

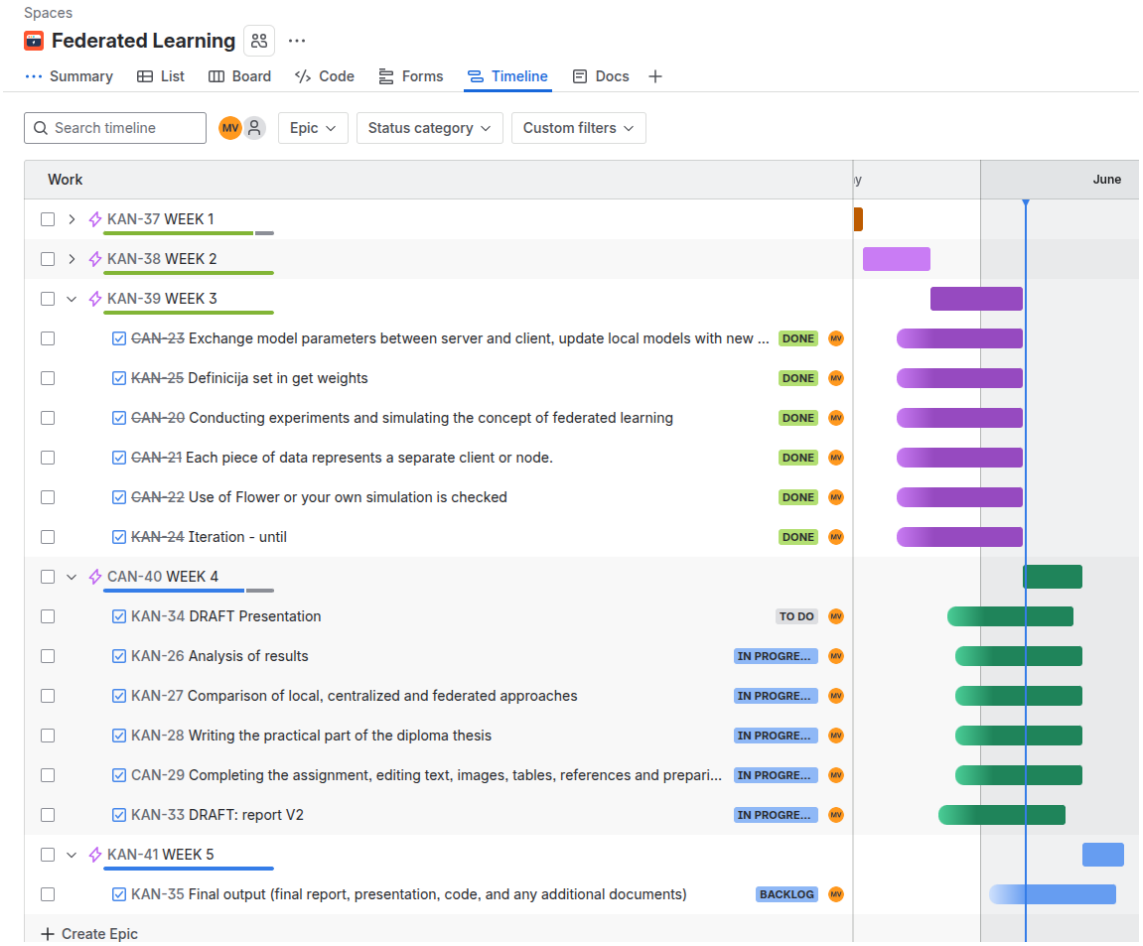
### A.1 Kanban board

Figure 2: Translated using Google Translate.



## A.2 Timeline

Figure 3: Translated using Google Translate.



## A.3 Backlog

Table 4: Original backlog of all work items with completion status. The tasks defined here are also a dissection of minimal viable requirements from Table **??**.

| key | workType | dueDate | weekNum | status | description |
|---|---|---|---|---|---|
| 30 | Task | 13/May/26 | Week 1 | Done | Follow-up details |
| 9 | Task | 17/May/26 | Week 1 | Done | Problem definition |
| 11 | Task | 17/May/26 | Week 1 | Done | Architecture definition |
| 14 | Task | 17/May/26 | Week 1 | Done | Introduction to Flower |
| 10 | Task | 17/May/26 | Week 1 | Done | Objective definition |
| 15 | Task | 17/May/26 | Week 1 | Done | Writing the theoretical part |
| 42 | Task | 17/May/26 | Week 1 | Done | Feedback about theoretical part |
| 12 | Task | 17/May/26 | Week 1 | Done | Development environment preparation |
| 13 | Task | 17/May/26 | Week 1 | Done | Overview of the use of models registered in MLFlow and the possibility of using the framework |
| 31 | Task | 22/May/26 | Week 2 | Done | DRAFT: Report V1 |
| 32 | Task | 22/May/26 | Week 2 | Done | ABSTRACT |
| 17 | Task | 25/May/26 | Week 2 | Done | Data flow preparation |
| 18 | Task | 25/May/26 | Week 2 | Done | Data division into three parts and integration with already built models in MLflow |
| 19 | Task | 25/May/26 | Week 2 | Done | Local models on three different data sets |
| 16 | Task | 25/May/26 | Week 2 | Done | Development of additional functionalities |
| 25 | Task | 05/Jun/26 | Week 3 | Done | Definition of set and get weights |
| 20 | Task | 05/Jun/26 | Week 3 | Done | Execution of experiments and simulation of the federated learning concept |
| 21 | Task | 05/Jun/26 | Week 3 | Done | Each part of the data represents a separate client or node |
| 22 | Task | 05/Jun/26 | Week 3 | Done | Verify the use of Flower or your own simulation |
| 24 | Task | 05/Jun/26 | Week 3 | Done | Iteration - until |
| 33 | Task | 10/Jun/26 | Week 4 | Done | DRAFT: report V2 |
| 34 | Task | 11/Jun/26 | Week 4 | Done | DRAFT Presentation |
| 26 | Task | 12/Jun/26 | Week 4 | Done | Result analysis |
| 28 | Task | 12/Jun/26 | Week 4 | Done | Writing the practical part of the thesis |
| 54 | Story | 12/Jun/26 | | Done | Single-machine and non-containerized simulation of apnea predictions using federated learning |
| 55 | Story | 12/Jun/26 | | Done | Single-machine and containerized (with Docker) simulation of apnea predictions using federated learning |
| 57 | Story | 12/Jun/26 | | Done | Multi-machine and containerized (with Docker) simulation of apnea predictions using federated learning |
| 56 | Story | 12/Jun/26 | | Not Done | Differential privacy mechanism implemented |
| 58 | Story | 12/Jun/26 | | Not Done | Multi-machine and containerized (on Kubernetes) simulation of apnea predictions using federated learning |

# B Second Test

Table 5: Results of centralized and federated learning approach and their comparison. In a federated setting, the test consisted of 3 hospitals (clients), 5 rounds and 15 epochs. In a centralized setting, it consisted of $15 \cdot 5 \cdot 3$ epochs (255 epochs). Hyperparameters were the same for both approaches. Number 0 or 1 after metric name indicates whether it was calculated for patients with or without obstructive sleep apnea.

| **Metric** | **Centralized** | **Federated** | **Difference** |
|---|---|---|---|
| train_accuracy | 0.80 | 0.80 | 0.00 |
| train_f1_1 | 0.49 | 0.52 | 0.03 |
| train_f1_0 | 0.80 | 0.77 | 0.03 |
| train_precision_1 | 0.44 | 0.42 | 0.02 |
| train_precision_0 | 0.84 | 0.87 | 0.03 |
| train_recall_1 | 0.57 | 0.68 | 0.11 |
| train_recall_0 | 0.76 | 0.69 | 0.07 |
| test_accuracy | 0.79 | 0.83 | 0.04 |
| test_precision | 0.39 | 0.00 | 0.39 |
| test_recall | 0.41 | 0.00 | 0.41 |
| avg_train_difference | | | 0.041 |
| avg_test_difference | | | 0.280 |
| avg_difference | | | 0.113 |